



Este documento constitui **enunciado da Sprint 2** do **Trabalho Prático em Grupo de Programação Modular**. **Leia-o com atenção** e até ao fim antes de começarem a realizar as tarefas.

O bom andamento do trabalho prático em grupo **depende da realização correta e no prazo de cada**, portanto fique atento às suas obrigações. **Alunos que deixarem de realizar entregas em 2 ou mais etapas** estão sujeitos a serem **excluídos dos grupos** e, portanto, ficam com **nota 0 no trabalho como um todo**.

Título do projeto: Xulamb's Parking

Finalizada a versão inicial, temos um aplicativo que consegue realizar as funções básicas pedidas pela *Xulamb's Inc.*: cadastrar estacionamentos e veículos; estacionar e liberar veículos; receber alguns relatórios simples. Podemos, então, avançar nos requisitos completos repassados pela empresa:

- As **vagas** dos estacionamentos são divididas em **tamanho pequeno, médio ou grande**. Desta maneira, também precisam ser cadastrados os **tamanhos dos veículos**, uma vez que estes **só podem usar vagas iguais ou maiores ao seu próprio tamanho**. A preferência, claro, é usar sempre uma vaga do tamanho exato do veículo;
- Os **estacionamentos** deverão ter **70% de vagas médias, 20% pequenas e 10% grandes**, espelhando os estudos da empresa sobre seus clientes em potencial. Caso necessário, **estes valores podem ser arredondados** (por exemplo, um estacionamento com 25 vagas pode ter 18 vagas grandes (72%), 5 pequenas e 2 grandes (8%));
- Para iniciar a **cobrança das vagas**, surge o conceito de “**Uso de vaga**”. Um **uso de vaga** representa uma **vaga** e um **veículo** associados por um **horário de entrada** e **outro de saída**. A cobrança básica é de **R\$4,50 a cada 15 minutos de uso**, sem tolerância em caso algum.
- Quando um veículo é **estacionado**, deve **criar** para si um novo **UsoDeVaga**. Ao ser **liberado**, **finaliza** o uso e **retorna o valor**.

Para refletir os requisitos, o modelo de classes do sistema evoluiu conforme a proposta que pode ser vista na última página deste documento. Seu grupo de trabalho não precisa, obrigatoriamente, seguir o modelo; porém, ele deve ser utilizado como guia para adequação do que não estiver funcionando bem no momento.

Tarefa 3 (tarefa de grupo):

- Os códigos das tarefas anteriores foram unificados e corrigidos pelo professor **na ramificação main** do seu repositório em https://classroom.github.com/a/FuBgBVR_. Observe as observações de correção e executem o aplicativo para verificar que os requisitos atuais estão funcionando corretamente.
- Lembre-se número designado a você em aula como **número de aluno** (de 1 a 5) para novas as tarefas de implementação.

Tarefa 4 (tarefa individual):

- **Crie uma ramificação**, a partir da *main* unificada, chamada **seuNome/Sprint2** (por exemplo, johnnyGreewood/Sprint2);
- De acordo com **seu número de aluno**, realize as **implementações** a seguir:
 1. Classe **UsoDeVaga**;
 2. Refatoração da classe **Veiculo** e suas heranças;
 3. Criar o enum **ETamanho**, seu método de extensão e alterar o aplicativo para pedir ao usuário qual tipo de veículo (carro, moto ou van) será criado no momento de cadastrar.
 4. Alterar o método **gerarVagas()** ou equivalente de **Estacionamento** para refletir a divisão dos tamanhos especificados de vagas;
 5. Refatoração da classe **Vaga** para incluir o **tamanho**, bem como **lançamento de exceções** nas suas operações com verificações incorretas;
- As tarefas devem ser, preferencialmente, feitas durante a aula. É obrigatório dar ao menos um **commit** na tarefa, com implementação significativa, durante a aula. Pendências podem ser resolvidas até 24h antes da próxima aula da disciplina, quando o professor realizará novo **merge** da *branch main*.

Prazos:

- **Esta etapa deve ser encerrada até 08/06.**

Instruções e observações:

- O projeto deve estar **obrigatoriamente hospedado** na tarefa do GitHub Classroom:
https://classroom.github.com/a/FuBgBVR_;
- **Atenção para as datas** das tarefas. As **avaliações** das atividades serão feitas **sempre** com o **último commit / pull request constante daquela data**.
- **Atenção para a política de uso de agentes de código e “inteligência artificial”** nas tarefas. Seu uso é **permitido, sob duas condições**:
 - O aluno deve **declarar**, no **cabeçalho** das classes, que **foi utilizado um agente de código**;
 - O aluno deve realizar **pelo menos dois commits**: o **primeiro** com o **código gerado automaticamente** e um **segundo** com as **revisões e melhorias realizadas** pelo aluno – as quais devem estar comentadas.
 - Em caso de **códigos com indícios de uso de agentes sem cumprir as condições acima**, a atividade terá sua **nota zerada, sem direito a recurso**.
- **A nota do aluno** em cada etapa é **vinculada às suas entregas individuais** e à nota de grupo. Alunos que **não cumprirem sua tarefa individual** ficam **sem nota**, mesmo que o grupo tenha cumprido as tarefas coletivas.

